Deep Learning Topics:
Special Connections Part IITransformers

Paul Rodriguez, PhD

(SDSC)

August2026

Outline

- Basic word prediction task and motivating the attention strategy
- A basic Attention Head network and exercise
- Transformers
- Transformers in Science Applications
- Options for starting deep learning development

Dependences of Language

Consider this sequence:

The Law will never be perfect, but it’s application should be just - this is what we are missing, in my opinion <End of Sequence>

What does ‘it’ refer to that can have an ‘application’?

Dependences of Language

Consider this sequence:

The Law will never be perfect, but it’s application should be just - this is what we are missing, in my opinion <End of Sequence>

What does ‘it’ refer to that can have an ‘application’?

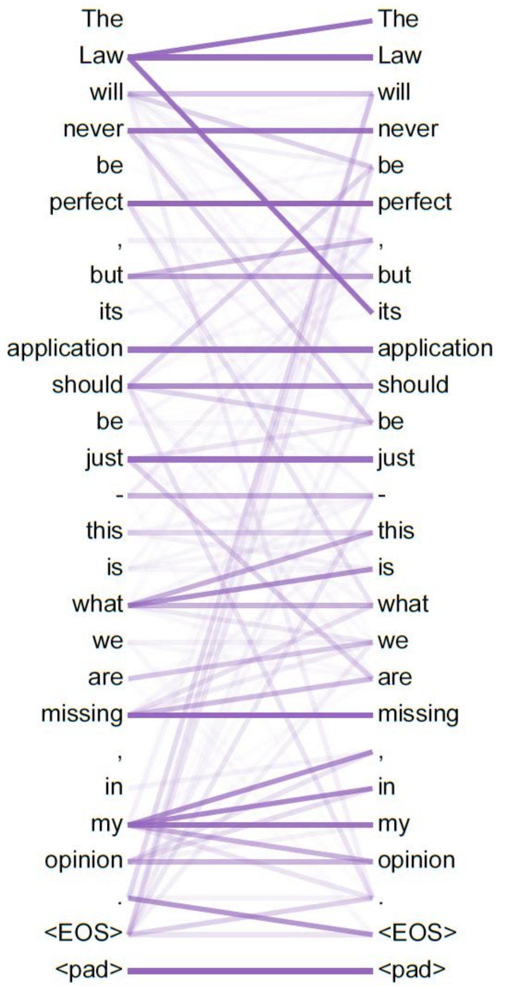
Dependences of Language

Consider this sequence:

The Law will never be perfect, but it’s application should be just - this is what we are missing, in my opinion <End of Sequence>

What does ‘it’ refer to that can have an ‘application’?

In language, there are many dependencies and interactions between words



A toy problem to get some intuition

- Let's use the following list of 7 tokens (ie vocabulary of 7words)

  1 <**s**tart>, 2 the, 3 man, 4 chicken, 5 ordered, 6 woman, 7 beef

- For input data, let's have these 2 sentences and token id sequences:

  <start> man ordered the chicken [1,3,5,2,4]
- <start> woman ordered the beef [1,6,5,2,7]

- **Now let's try to predict the next word by 'attention' idea**

The toy task: predict next word

Predict the next word for the first input sentence after each word:

<Start> man ordered the chicken

The toy task: predict next word

Predict the next word for the first input sentence after each word:

<Start> man ordered the chicken

A basic solution is a bigram matrix:

X=Sequence-to-Word, size is TxV

Next Token (ie word) Prediction

Token Sequence

Pos	Word	<strt>	The	Man	Chik	Ord	Wmn	Beef
1	<start>			0.5			0.5	
2	Man					1.0		
3	Orde.r		1.0					
4	The				0.5			0.5
5	Chick.	1.0						

The toy task: predict next word

Predict the next word for the first input sentence after each word:

<Start> man ordered the chicken

A basic solution is a bigram matrix:
X=Sequence-to-Word, size is TxV

Next Token (ie word) Prediction

Token Sequence

Pos	Word	<strt>	The	Man	Chik	Ord	Wmn	Beef
1	<start>			0.5			0.5	
2	Man					1.0		
3	Orde.r		1.0					
4	The				0.5			0.5
5	Chick.	1.0						

The bigram doesn't use context

The toy task: predict next word

Predict the next word for the first input sentence after each word:

<Start> man ordered the chicken

A basic solution is a bigram matrix:
X=Sequence-to-Word, size is TxV

Challenge, can we learn predictions (→) that depend on context of other tokens and/or position

After ‘Man’ the→ chicken = 1.0

After ‘Woman’ the → beef = 1.0

nce

Next Token (ie word) Prediction

Pos	Word	<strt>	The	Man	Chik	Ord	Wmn	Beef
1	<start>			0.5			0.5	
2	Man					1.0		
3	Orde.r		1.0					
4	The				0.5			0.5
5	Chick.	1.0						

The toy task: predict next word

Predict the next word for the first input sentence after each word:

<Start> man ordered the chicken

A basic solution is a bigram matrix:

X=Sequence-to-Word, size is TxV

Challenge, can we learn predictions (→) that depend on context of other tokens and/or position

After   'Man'   the → chicken = 1.0

After   'Woman'   the → beef = 1.0

nce

Next Token (ie word) Prediction

Pos	Word	<strt>	The	Man	Chik	Ord	Wmn	Beef
1	<start>			0.5			0.5	
2	Man					1.0		
3	Orde.r		1.0					
4	The				0.5			0.5
5	Chick.	1.0						

We want 'man' context to pass information

The attention idea

Let's get all tokens to 'pass information' about dependencies

E.G. for *X a TxV* matrix of possible predictions, we want to transform *X into contextualized predictions*

X= Sequence-to-Word is TxV

$$\begin{pmatrix} 0 & 0 & 0.5 & 0 & 0 & 0.5 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0.5 & 0 & 0 & 0.5 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix} \Rightarrow$$

Contextualized Predictions

The attention idea

Let's get all tokens to 'pass information' about dependencies

E.G. for *X a TxV* matrix of possible predictions, we want to transform *X into contextualized predictions*

We need a W a TxT matrix – aka 'attention' weights that has word dependencies

X= Sequence-to-Word is TxV

$$\begin{pmatrix} 0 & 0 & 0.5 & 0 & 0 & 0.5 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0.5 & 0 & 0 & 0.5 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix} \Rightarrow$$

Contextualized Predictions

The attention idea

Let's get all tokens to 'pass information' about dependencies

E.G. for *X a TxV* matrix of possible predictions, we want to transform *X into contextualized predictions*

We need a W a TxT matrix – aka 'attention' weights that has word dependencies

X= Sequence-to-Word is TxV **Contextualized Predictions**

$$W_{TxT} * \begin{pmatrix} 0 & 0 & 0.5 & 0 & 0 & 0.5 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & \boxed{0.5 \quad 0 \quad 0 \quad 0.5} \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix} \Rightarrow \begin{pmatrix} 0 & 0 & 0.5 & 0 & 0 & 0.5 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & \boxed{1 \quad 0 \quad 0 \quad 0} \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix}$$

The attention idea

Let’s get all tokens to ‘pass information’ about dependencies

E.G. for *X a TxV* matrix of possible predictions, we want to transform *X into contextualized predictions*

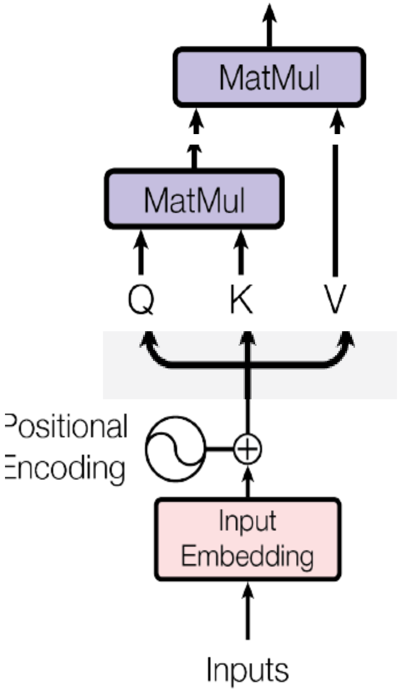
We need a W a TxT matrix – aka ‘attention’ weights that has word dependencies

X= Sequence-to-Word is TxV **Contextualized Predictions**

$$W_{TxT} * \begin{pmatrix} 0 & 0 & 0.5 & 0 & 0 & 0.5 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0.5 & 0 & 0 & 0.5 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix} \Rightarrow \begin{pmatrix} 0 & 0 & 0.5 & 0 & 0 & 0.5 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix}$$

W should reflect the interdependencies of the words in the input
Q: Where should W come from? (Hint: what does all AI need)

- **Let's build up the attention architecture**



Get input embeddings

First, get sequence of token embeddings using 1-hot input to E units – see next slide

[1,3,5,2,4] → X_{TxE}

Sidenote: embedding (encoding) for words (ie tokens)

Each word (or word part) is assigned a ‘token id’ 1 to V=50K

Let X be a “1-hot” vector (only the token id-th element is 1)

X is Vx1

0
0
.
.
.
0
1
0
.
.
.
0
0

Sidenote: embedding (encoding) for words (ie tokens)

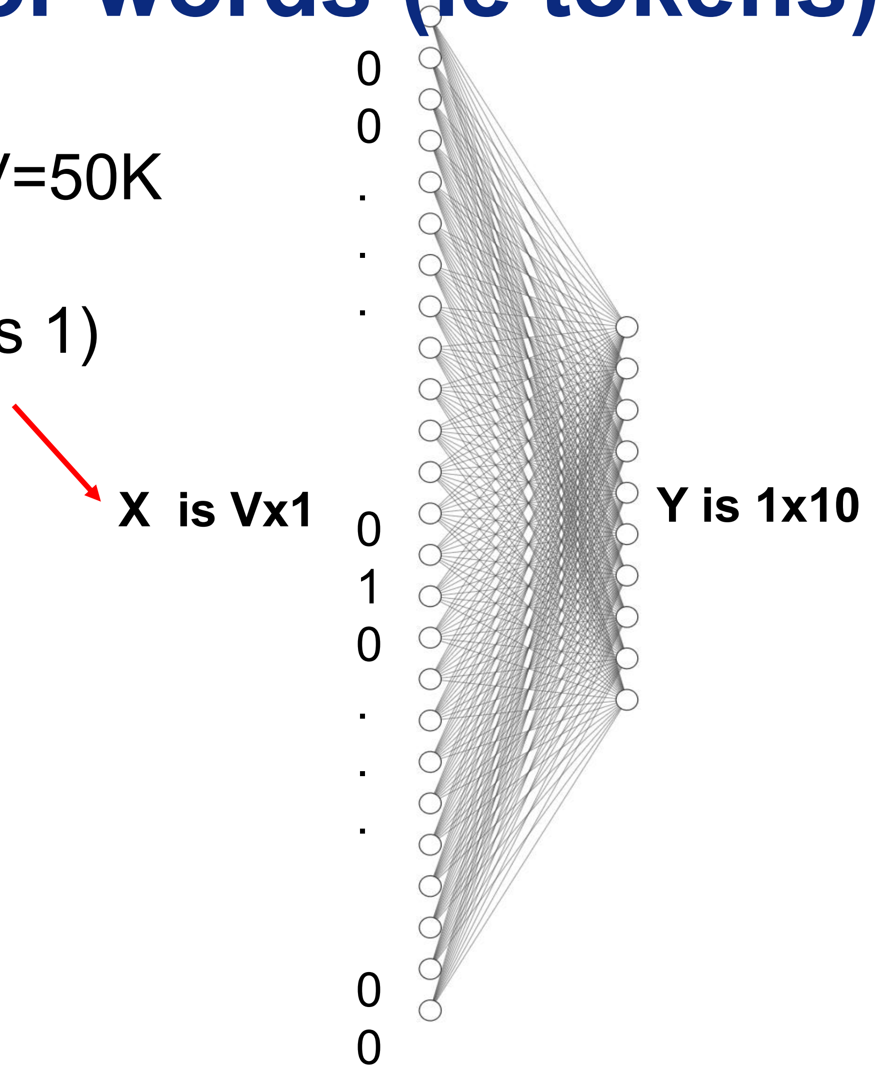
Each word (or word part) is assigned a 'token id' 1 to V=50K

Let X be a "1-hot" vector (only the token id-th element is 1)

Use a one layer network to make an embedding

In this picture, each token id is converted to a lower dimensional vector with size 1x10

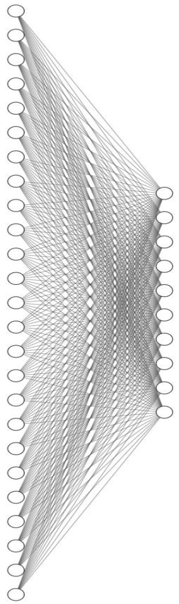
Each token sequence of T vectors is Tx10



(back to) Get input embeddings

First, get sequence of token embeddings using 1-hot input to E units

[1,3,5,2,4] → X_{TxE}



Get input and add in position info

First, get sequence of token embeddings using 1-hot input to E units

[1,3,5,2,4] → X_{TxE}

Then do the same for positions 1…T

[1,2,3,4,5] → P_{TxE}

Get input and add in position info

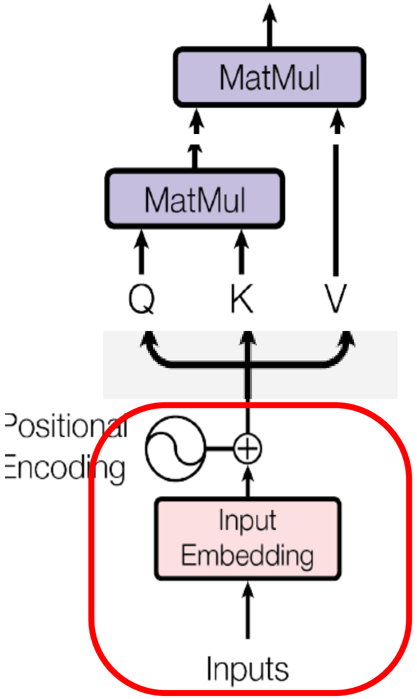
First, get sequence of token embeddings using 1-hot input to E units

[1,3,5,2,4] → X_{TxE}

Then do the same for positions 1…T

[1,2,3,4,5] → P_{TxE}

X + P is final TxE matrix of input embeddings



Embeddings for attention weights

Take Input Embeddings and build a 'Query' (Q) and 'Key' (K) embedding matrix *of size TxH*

$X + P \rightarrow \quad Q_{TxH}$

$X + P \rightarrow \quad K_{TxH}$

The idea is to make new embeddings that can useful for word-to-word dependencies

Embeddings for attention weights

Take Input Embeddings and build a 'Query' (Q) and 'Key' (K) embedding matrix *of size TxH*

$X + P \rightarrow Q_{TxH}$

$X + P \rightarrow K_{TxH}$

The idea is to make new embeddings that can useful for word-to-word dependencies

Now let $W = Q*K'$

Notice that every token's embedding gets to 'interact' with every other token's embedding to make up the *TxT* elements of ***W***

Embeddings for attention weights

Take Input Embeddings and build a ‘Query’ (Q) and ‘Key’ (K) embedding matrix *of size TxH*

$X + P \rightarrow Q_{TxH}$

$X + P \rightarrow K_{TxH}$

The idea is to make new embeddings that can useful for word-to-word dependencies

Now let $W = Q*K'$

Notice that every token’s embedding gets to ‘interact’ with every other token’s embedding to make up the *TxT* elements of ***W***

$X + P \rightarrow V_{TxV}$

Finally, instead of a pre-built bigram matrix, use another embedding for a ‘Value’ V matrix

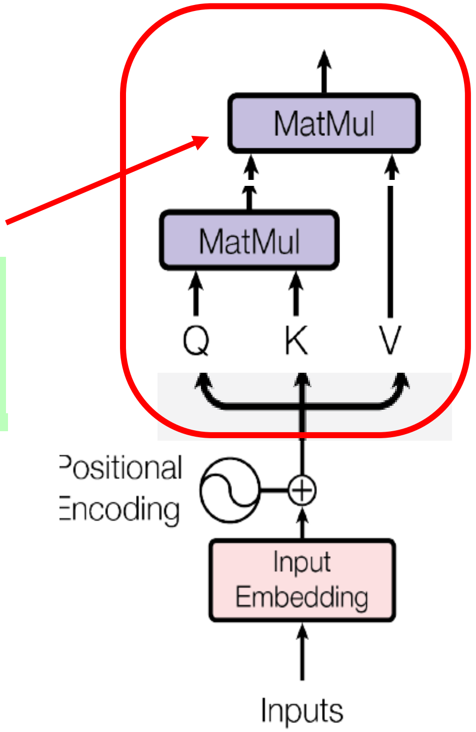
Embeddings for attention weights

Take Input Embeddings and build a ‘Query’ (Q) and ‘Key’ (K) embedding matrix *of size TxH*

$X + P \rightarrow Q_{TxH}$

$X + P \rightarrow K_{TxH}$

Last, we can take sigmoid of W * V to get TxV predictions from 0 to 1



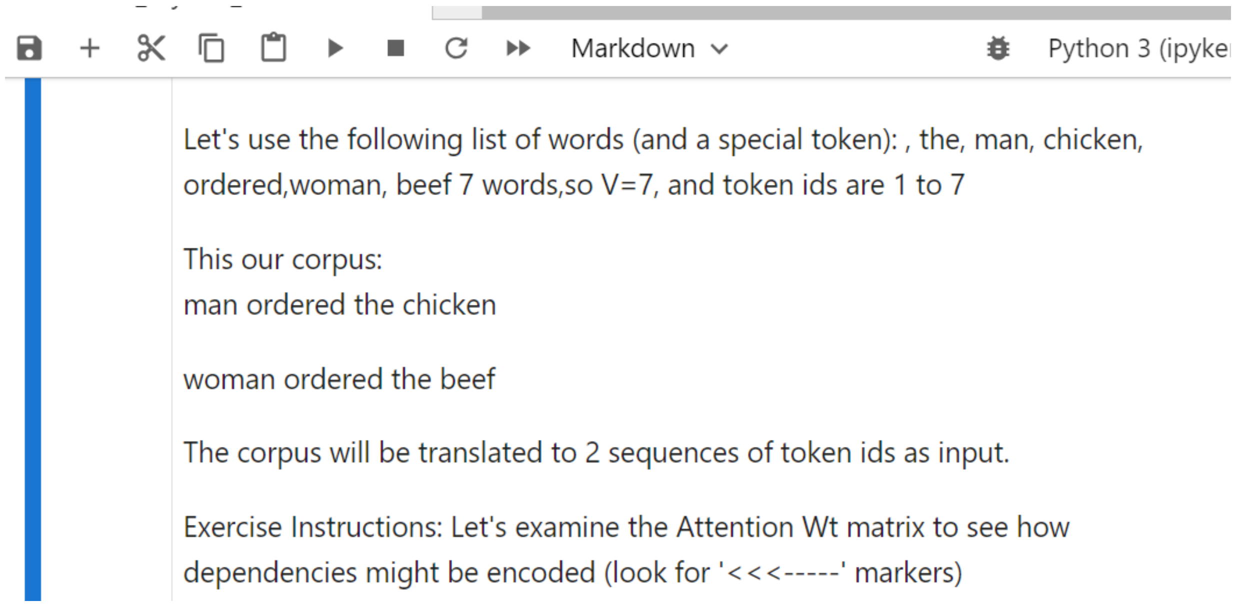
Now let $W = Q*K'$

$X + P \rightarrow V_{TxV}$

In class exercise:

An example of attention head with a toy task:

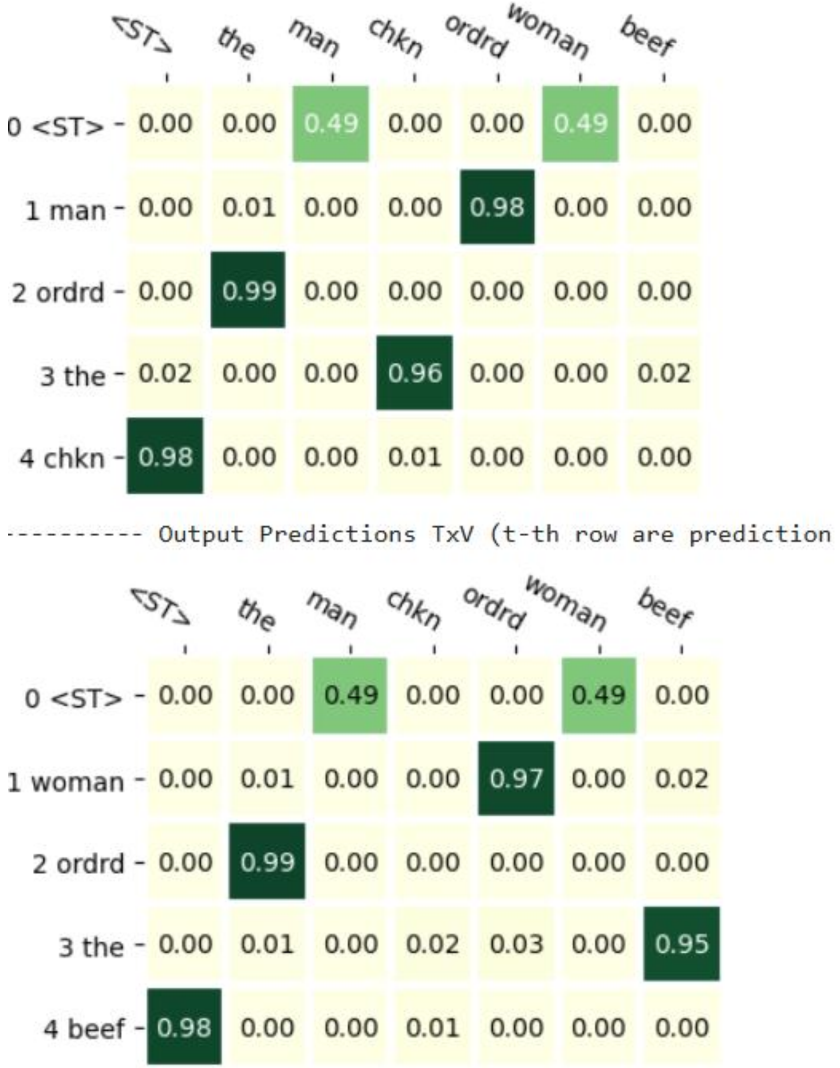
Run the “toytask_attention notebook” and observe the printed predictions and attention weights.



Output TxV predictions

Notice <start> → [man or woman at 0.50]

Notice that the→ [chicken or beef] predictions change depending on who’s ordering



Here are the W (attention) and V (value) matrices for [<st> man ordrd the chckn]

Notice the upper diagonal of W are masked so that predictions don't "look ahead"

Notice the <Start> token only depends on itself, and only picks off top row of V

	0 <ST>	1 man	2 ordrd	3 the	4 chkn
0 <ST>	1.00	0.00	0.00	0.00	0.00
1 man	0.06	0.94	0.00	0.00	0.00
2 ordrd	0.20	0.52	0.29	0.00	0.00
3 the	0.01	0.24	0.71	0.05	0.00
4 chkn	0.25	0.01	0.07	0.66	0.01

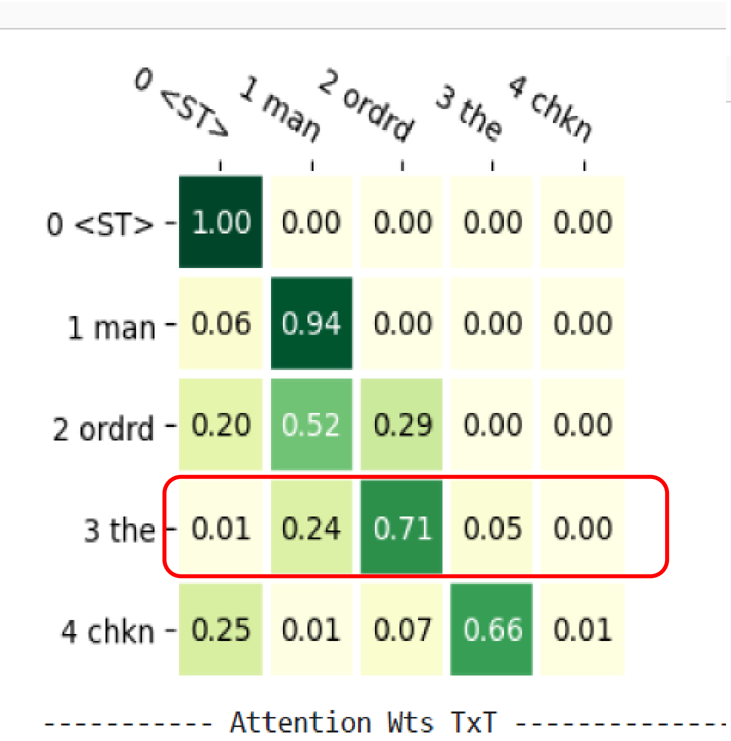
----------- Attention Wts TxT -------------

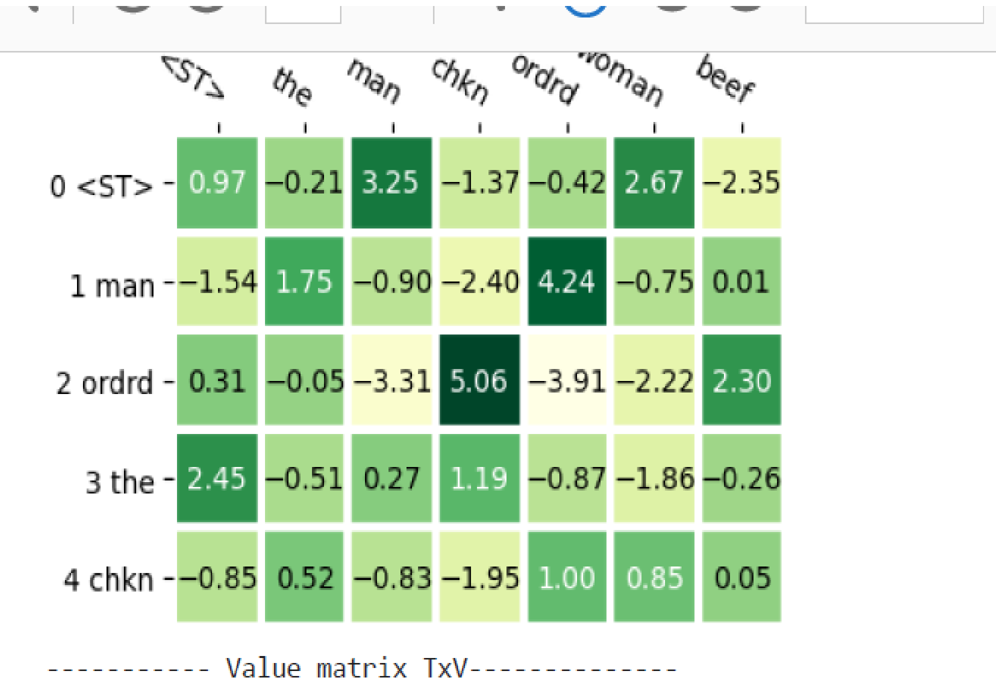
	<ST>	the	man	chkn	ordrd	woman	beef
0 <ST>	0.97	−0.21	3.25	−1.37	−0.42	2.67	−2.35
1 man	−1.54	1.75	−0.90	−2.40	4.24	−0.75	0.01
2 ordrd	0.31	−0.05	−3.31	5.06	−3.91	−2.22	2.30
3 the	2.45	−0.51	0.27	1.19	−0.87	−1.86	−0.26
4 chkn	−0.85	0.52	−0.83	−1.95	1.00	0.85	0.05

----------- Value matrix TxV-------------

Here are the W (attention) and V (value) matrices for [<st> man ordrd the chckn]

The 4th row of W 1xT times V makes the 1xV predictions of ‘the’

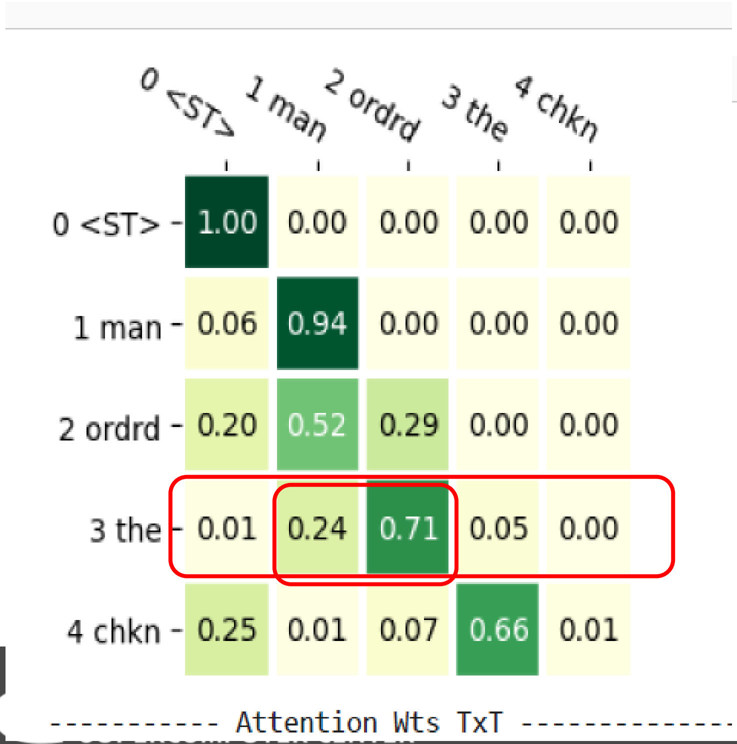


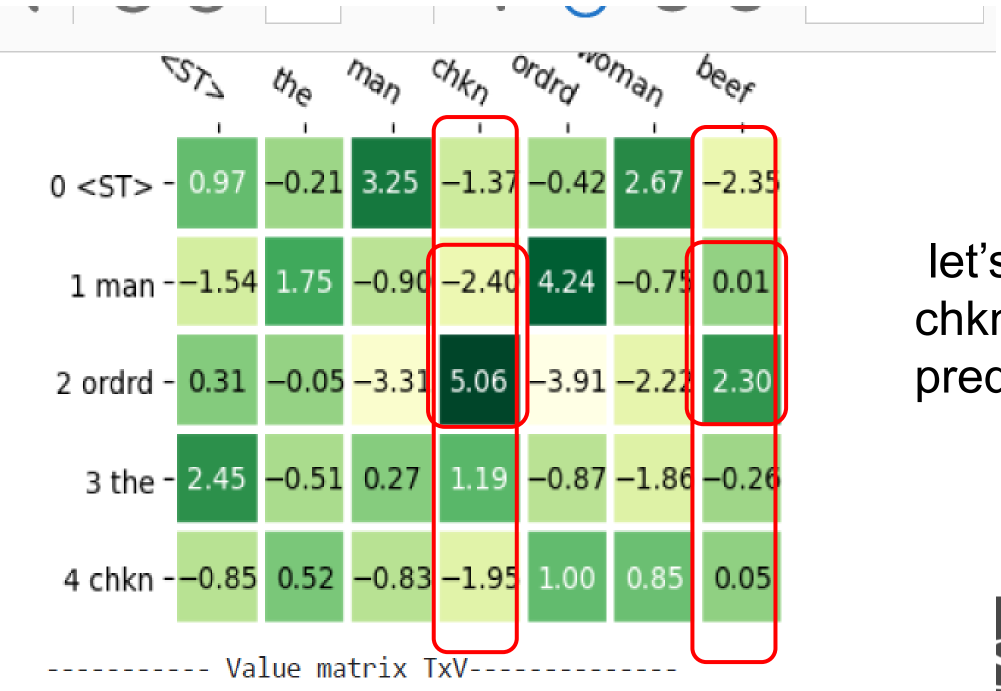


Here are the W (attention) and V (value) matrices for [<st> man ordrd the chckn]

The 4th row of W 1xT times V makes the 1xV predictions of ‘the’

only 2 values in 4th row > 0





let’s focus on chkn, beef predictions

Here are the W (attention) and V (value) matrices for [<st> **wmn** ordrd the **beef**]:

The 4^{th} row of W 1xT times V makes the 1xV predictions of 'the'

only 2 values in 4^{th} row > 0

----------- Attention Wts TxT -------------

	0 <ST>	1 woman	2 ordrd	3 the	4 beef
0 <ST>	1.00	0.00	0.00	0.00	0.00
1 woman	0.01	0.99	0.00	0.00	0.00
2 ordrd	0.15	0.62	0.22	0.00	0.00
3 the	0.00	0.52	0.45	0.03	0.00
4 beef	0.26	0.00	0.07	0.66	0.01

----------- Value matrix TxV-------------

	<ST>	the	man	chkn	ordrd	woman	beef
0 <ST>	0.97	−0.21	3.25	−1.37	−0.42	2.67	−2.35
1 woman	−1.90	2.02	−1.95	−2.84	4.43	−1.68	0.97
2 ordrd	0.31	−0.05	−3.31	5.06	−3.91	−2.22	2.30
3 the	2.45	−0.51	0.27	1.19	−0.87	−1.86	−0.26
4 beef	−0.91	0.45	−0.79	−1.98	1.03	0.89	0.05

let's focus on chkn, beef predictions

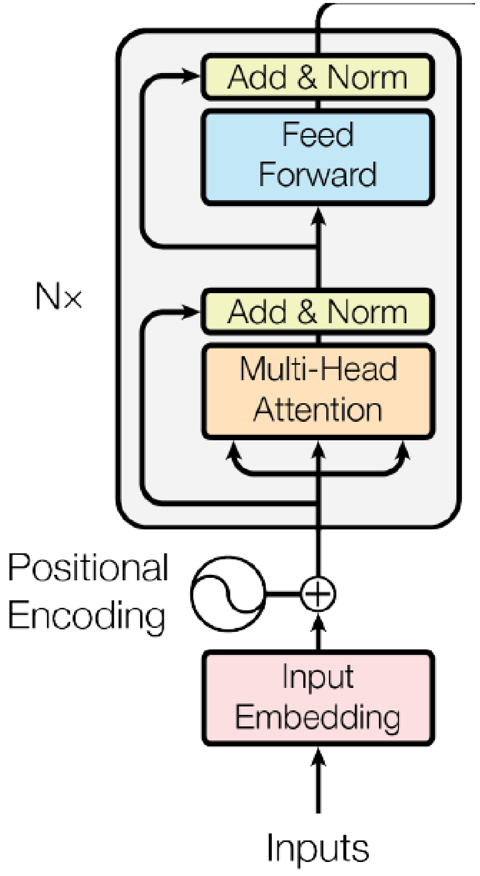
Pause –

- Let scale up to transformers

Finally, a transformer

Include skip-add connections

Include Layer Normalization or DropOut layers

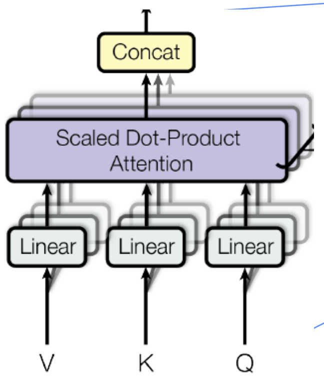


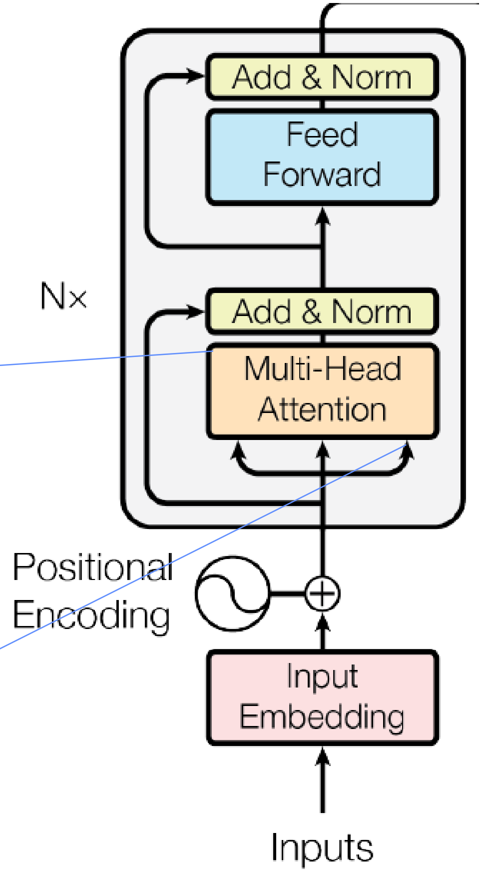
Finally, a transformer

Include skip-add connections

Include Layer Normalization or DropOut layers

Multi-Head – for N heads produce *Tx(H/N)* each





Finally, a transformer

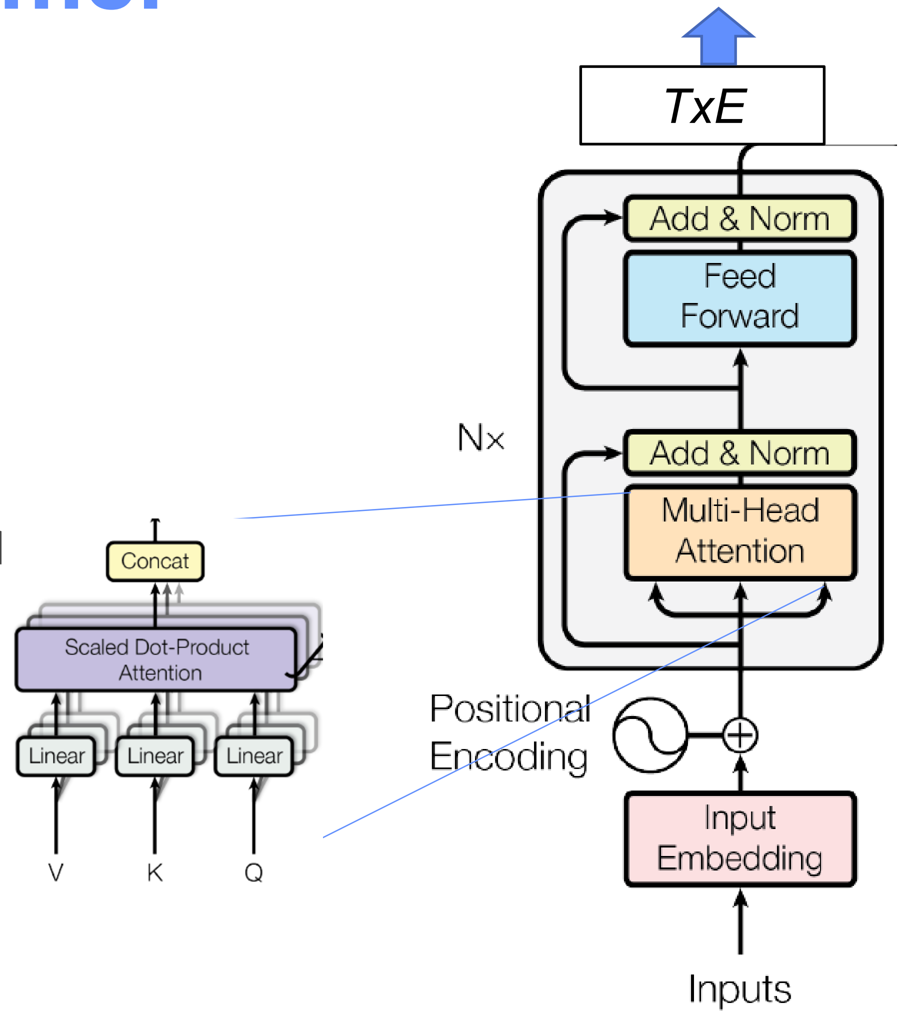
Include skip-add connections

Include Layer Normalization or DropOut layers

Multi-Head – for N heads produce *Tx(H/N)* each

Add MLP layers on top –

output another *TxE* matrix



Finally, a transformer

Include skip-add connections

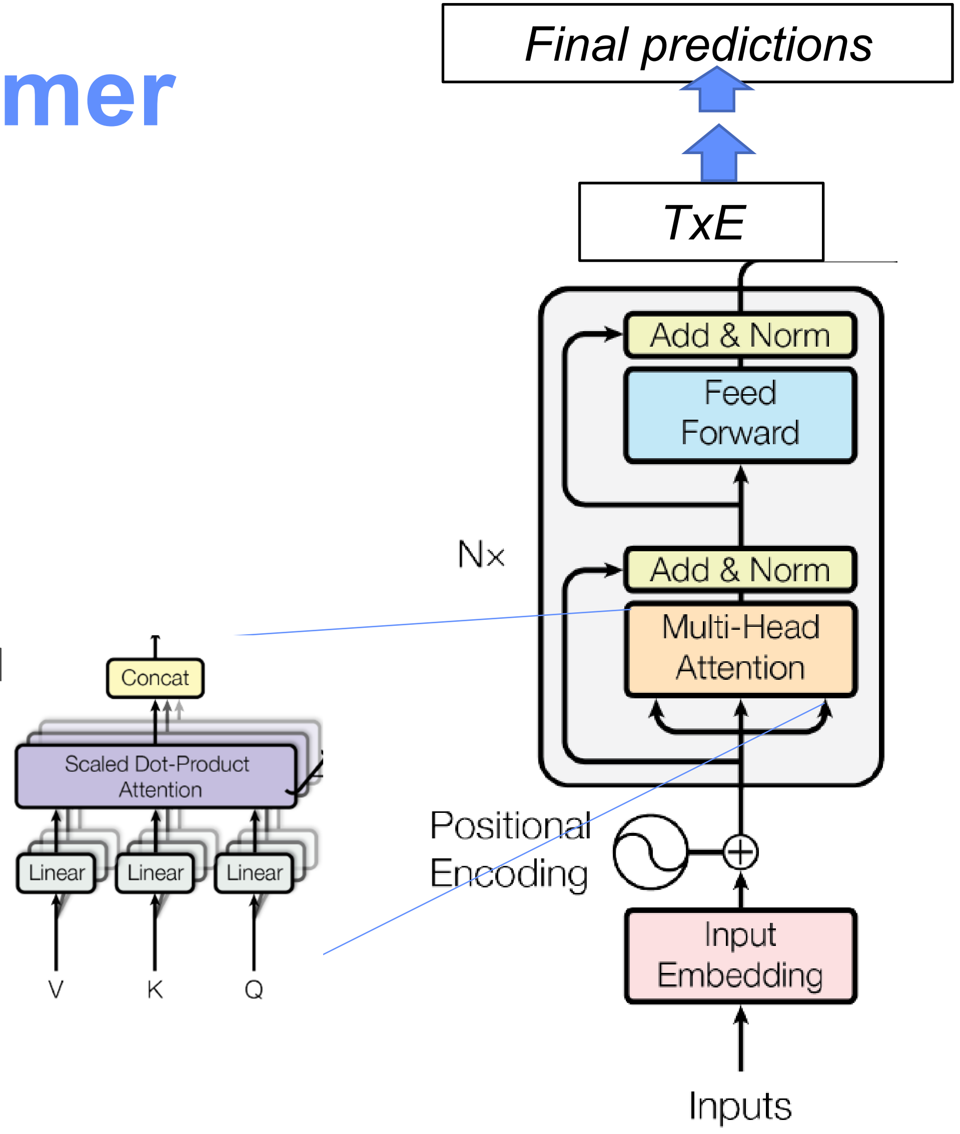
Include Layer Normalization or DropOut layers

Multi-Head – for N heads produce *Tx(H/N)* each

Add MLP layers on top –

output another *TxE* matrix

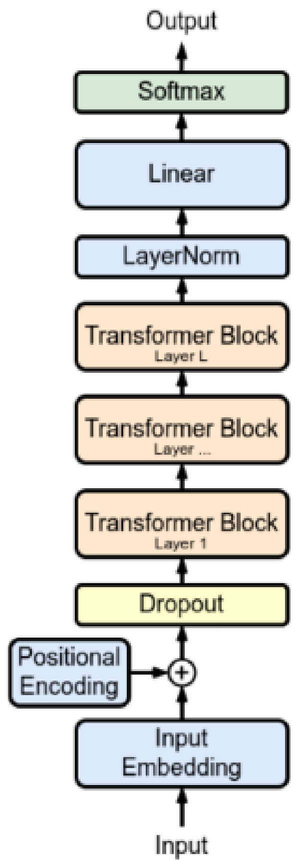
or output final probabilities



2 kinds of training strategies

GPT – predict next word, use mask to only look back at prior context (which could be a whole document)

BERT – predict words and other targets, no mask



- **Transformers for Science applications**

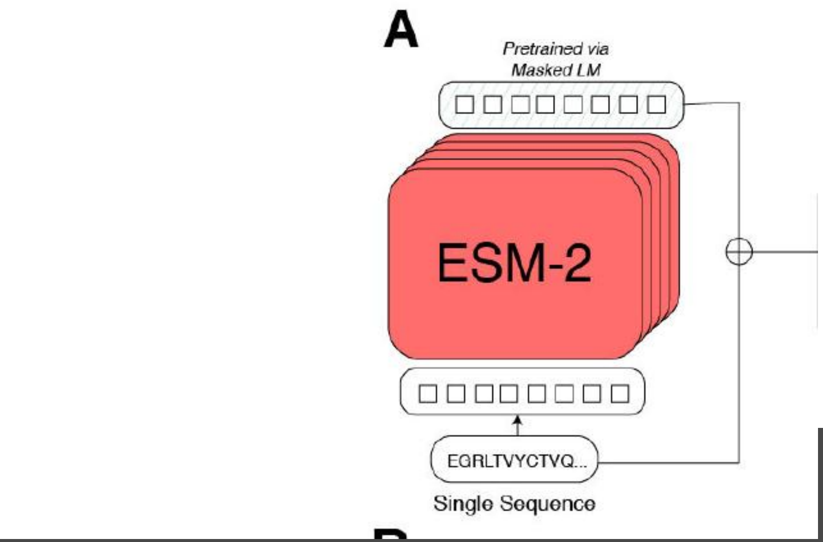
Can anything be cast as a kind of sentence, or an arrangement of tokens?

ESM Fold model

Language models of protein sequences at the scale of evolution enable accurate structure prediction

Lin etal, Meta Research 2022

- Atom level structure prediction
- Uses protein sequence as input to transformer layers (like LLM)



ESM Fold model

Language models of protein sequences at the scale of evolution enable accurate structure prediction

Lin etal, Meta Research 2022

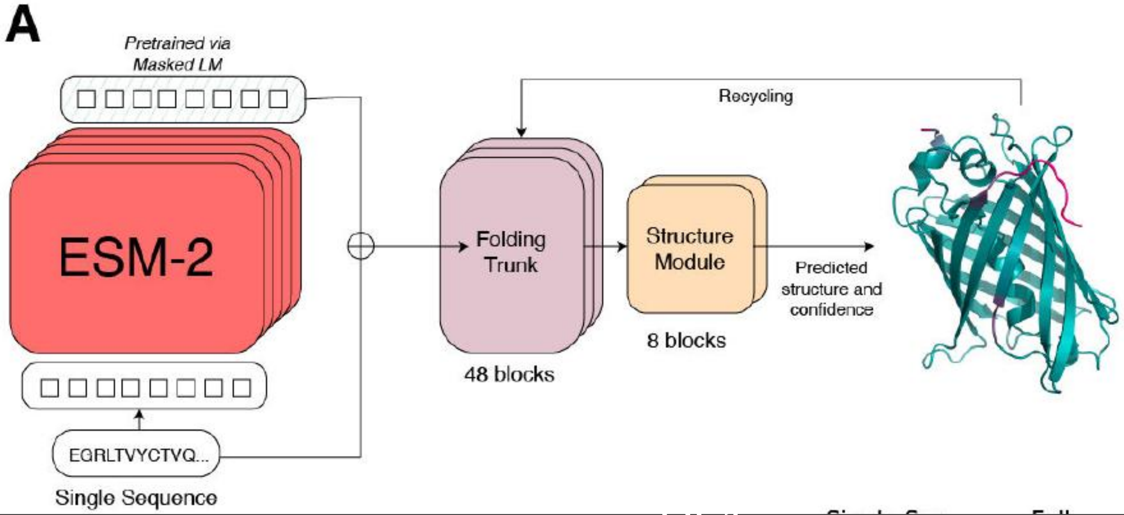
- Atom level structure prediction
- Uses protein sequence as input to transformer layers (like LLM) then ***feed Attention Matrix*** to 'folding block'
- Iteratively refine a predicted map of protein contact

(similar to AlphaFold2, but faster)

(a) KinB (b) Spo0E

Figure 1. Protein Structures.

The two protein structures presented here KinB and Spo0E were generated by providing sequence of ~300 amino acids to ESMFold. (M.Gujral SDSC)



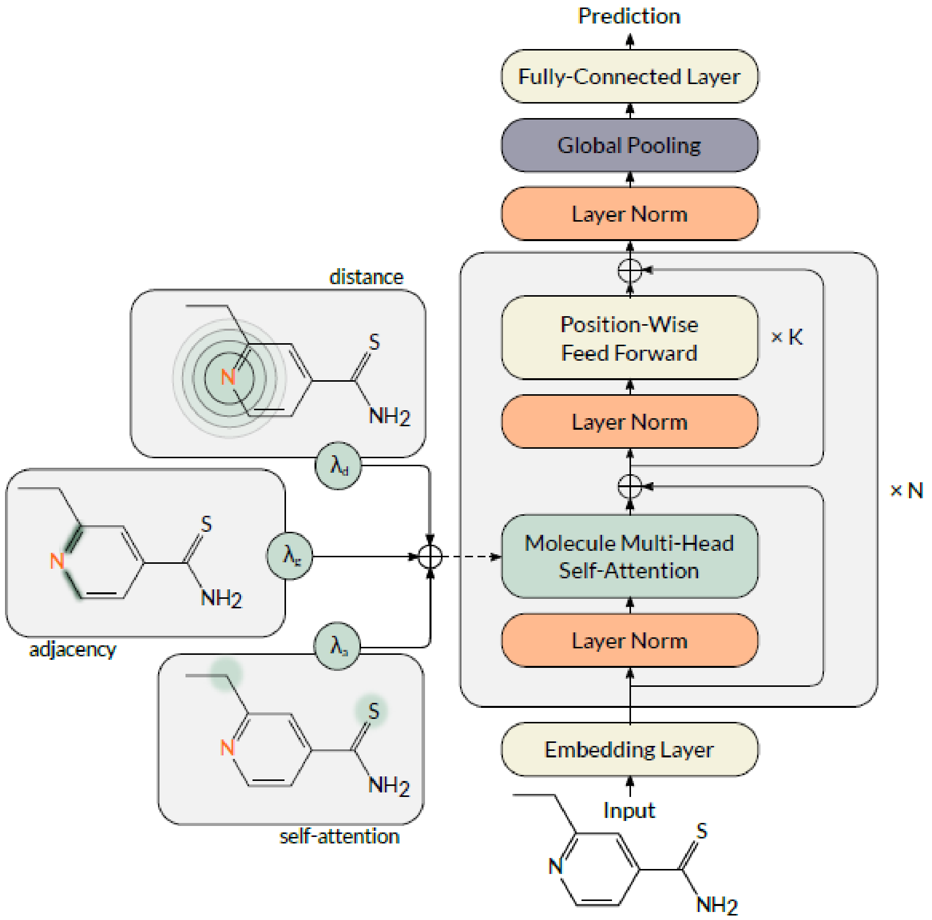
Molecule Attention Transformer

(Maziarka et al. 2020)

Molecular property prediction

Uses the set of atoms as input (like sentence tokens)

Includes spatial information by using a sum of the attention matrix, a distance matrix, and an adjacency matrix.



The Visual Transformer (ViT)

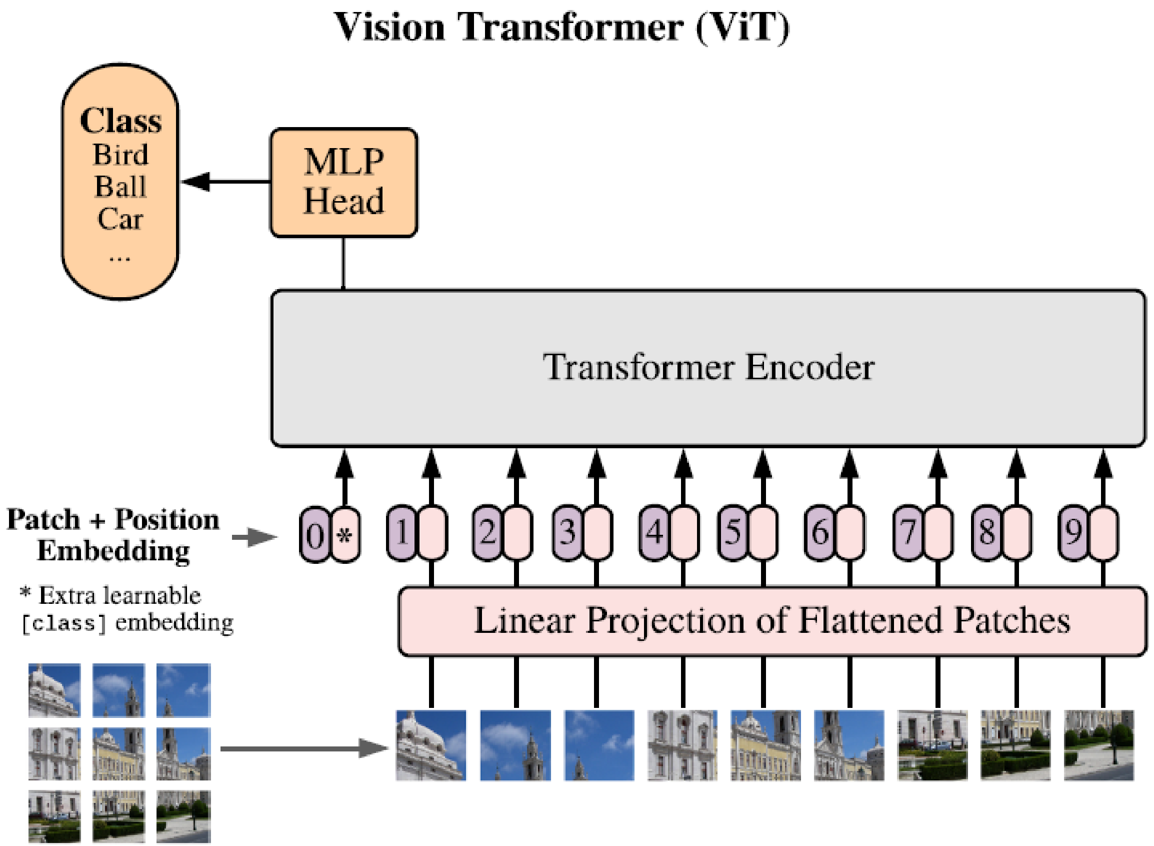
An image is worth 16x16 words: Transformers for image recognition at scale

Adosovitski, et al, 2021, Google Research

Uses a sequences of image patches (16x16) like a sentence of tokens (ie 224x224 pixels is 16x16 patches of 14x14 pixels)

Uses a classification token like Bert to learn image output classes

Competitive or better than CNNs but might need more data



- **Combining images and text often makes DL work better, or more generic, for image or text tasks**

CLIP – Contrastive Language-Image Pretraining

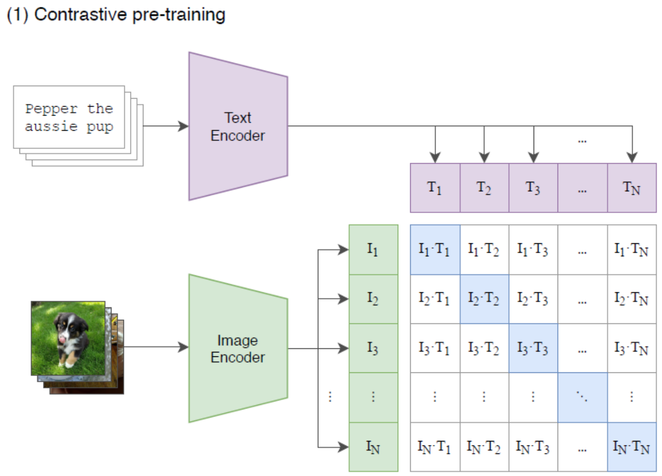
Learning Transferable Visual Models From Natural Language Supervision

Radford et al, 2021, Open AI

Uses 400M images and captions for training

Learns a multi-modal embedding

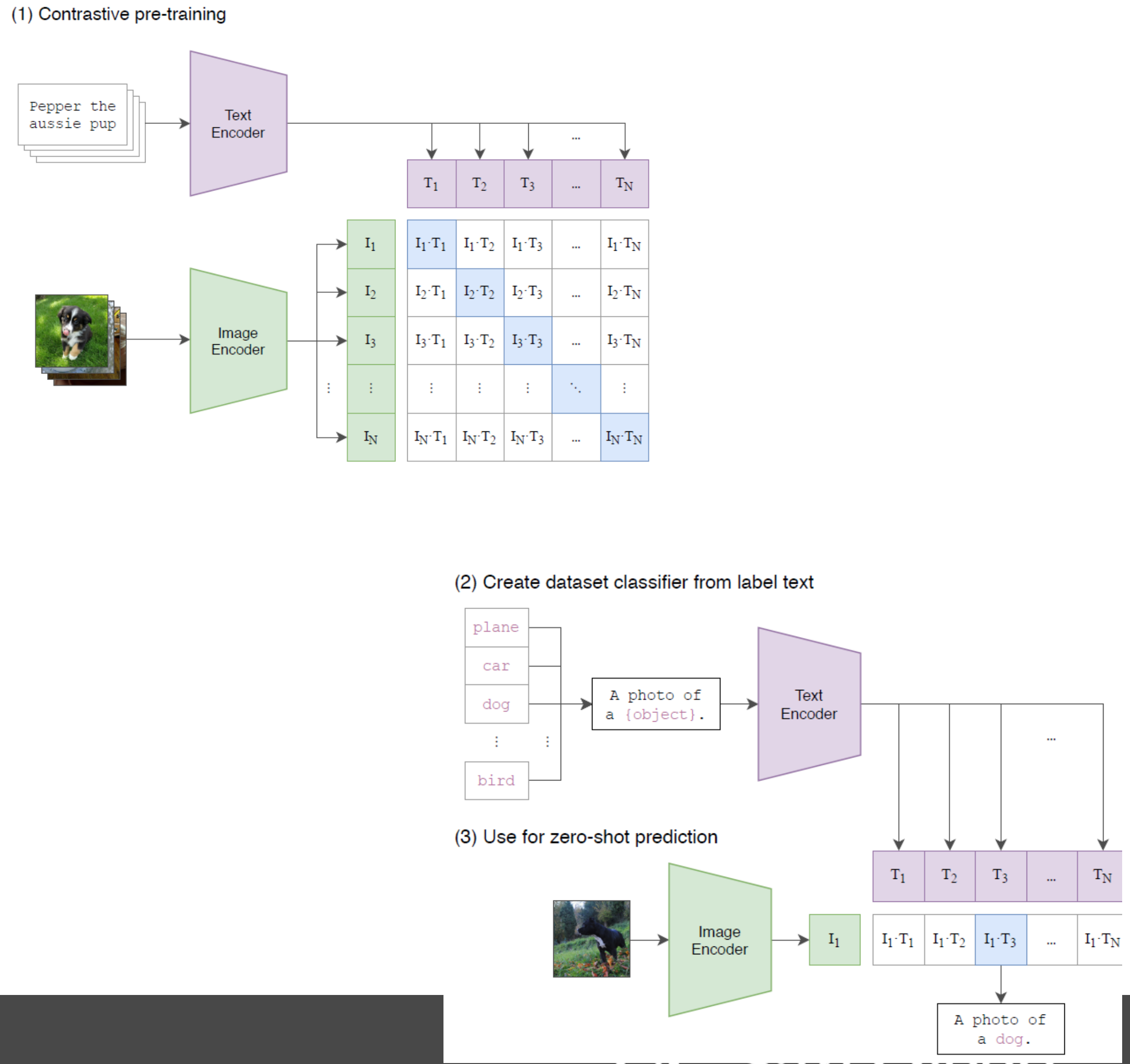
Maximizes embedding similarity of captions with it's image; minimizes embedding similarity of captions with other images



CLIP – Contrastive Language-Image Pretraining

Learning Transferable Visual Models From Natural Language Supervision

Radford et al, 2021, Open AI



Uses 400M images and captions for training

Learns a multi-modal embedding

Maximizes embedding similarity of captions with it's image; minimizes embedding similarity of captions with other images

Performs classification by prompting it with an image and possible captions

Note: CLIP with diffusion gets close to DALL-E

How to start?

- Assuming you have data, task, and/or model ideas, here are 4 general options:

1. Code from scratch with pytorch functions - you get total control

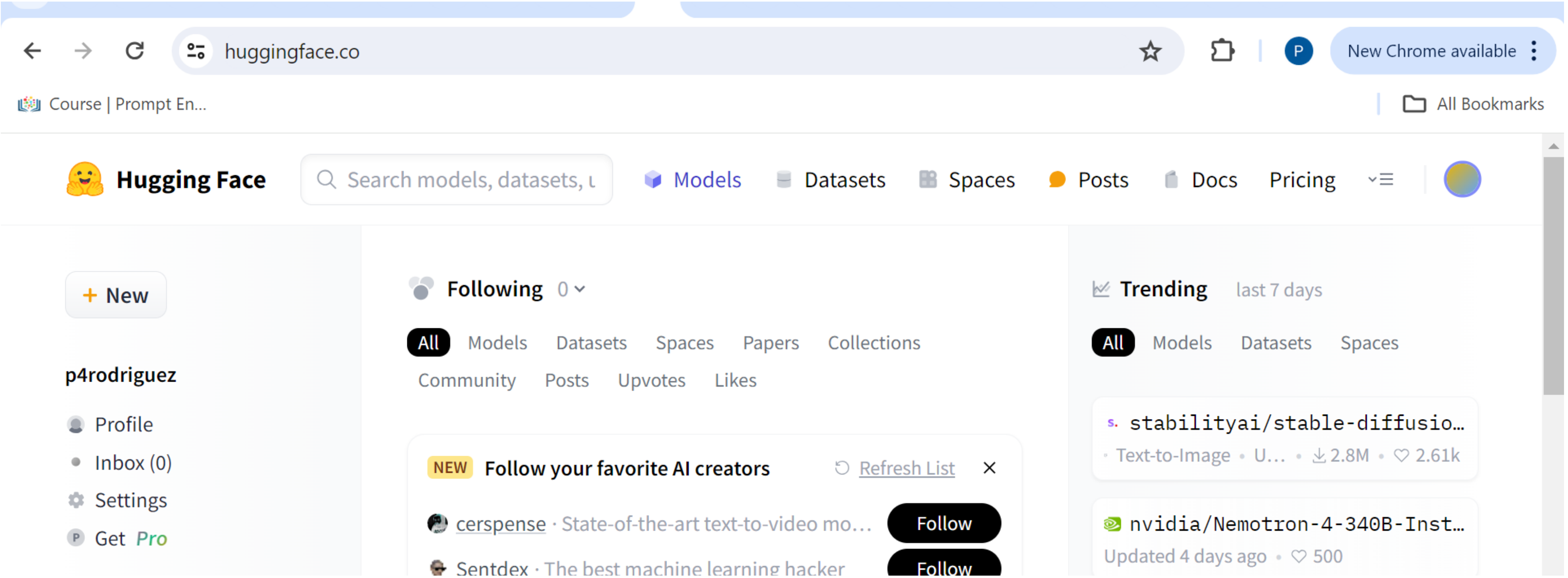
2. Code from some pre-configured models with your own customizations - not all customizations are easy

3. Fine-tune pre-trained model – your task must be similar enough

4. Download a package - your task has to be same

Hugging Face Hub

- huggingface.com is a hub of models, data, tutorials for using AI models



Using Hugging Face (HF)

- HF provides python packages to make it (relatively) easy to run models
- May require an HF authentification token
- Some of the main packages are:

pipeline:	to run inferencing
diffusers:	pretrained diffusion models
transformers:	utilities for text processing
datasets:	to access data from hub

Replicating Toy Attention task:

- A GPT2-like model is specified by a configuration
- Declare the model and it’s ready for training

```
from transformers import GPT2LMHeadModel, GPT2Config

#  ------------- from the toy attention head task -------
V = 7   # GPT2 model adds another start token  (and I dont' want
E = 32
T = 5

# Small config, randomly initialized (not pretrained)
config = GPT2Config(
    attn_implementation="eager", #save attention wts
    vocab_size=V,
    n_positions=T,
    n_embd=E,
    n_layer=1,
    n_head=1,
)

model = GPT2LMHeadModel(config);
```

Take home task

- Run the “hugging face gpt2 notebook”
- Training loop is a few lines of code
- the predictions and attention weights will look similar to the toy attention task

```
[4]: optimizer = torch.optim.AdamW(model.parameters(), lr=1e-4) # lr=1e-3)

     for step in range(1500):
         optimizer.zero_grad()
         outputs = model(InSet, labels=TarSet)
         loss = outputs.loss
         loss.backward()
         optimizer.step()

         if step % 20 == 0:
             print(f"Step {step}: loss = {loss.item():.4f}")
```

```
`loss_type=None` was set in the config but it is unrecognized. Using the defau
Loss`.
Step 0: loss = 2.0747
Step 20: loss = 1.8696
Step 40: loss = 1.6405
Step 60: loss = 1.5368
```

end